

Aprendizaje de Máquina

Sesión 2: Tipos de aprendizaje y primer notebook

Angel Rodríguez

Objetivos de la sesión

- ▶ Distinguir entre aprendizaje supervisado, no supervisado y por refuerzo.
- ▶ Reconocer problemas de clasificación, regresión y agrupamiento.
- ▶ Identificar los elementos básicos de un conjunto de datos: observaciones, variables, etiquetas y características.
- ▶ Comprender la idea de entrenamiento, prueba y evaluación inicial de un modelo.
- ▶ Ejecutar un primer notebook en Python usando Scikit-Learn.

Agenda de la sesión

1. Repaso breve de la sesión anterior.
2. Tipos de aprendizaje de máquina.
3. Clasificación, regresión y agrupamiento.
4. Estructura de un conjunto de datos.
5. Primer flujo de trabajo con Scikit-Learn.
6. Actividad en notebook.

Idea central

Antes de programar modelos complejos, necesitamos saber **qué tipo de problema** estamos resolviendo.

Repaso: ¿qué es aprendizaje de máquina?

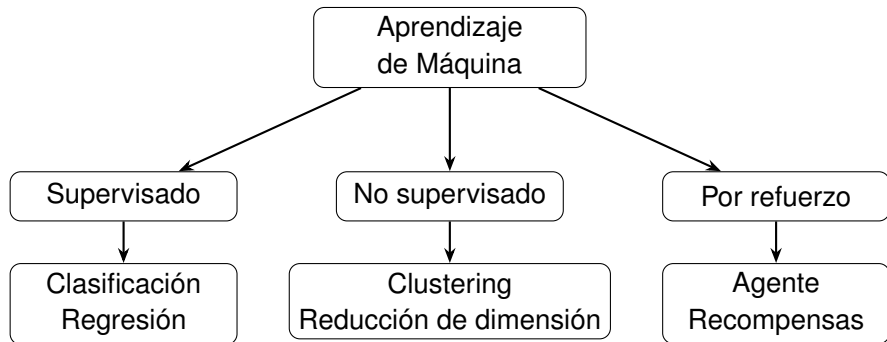
En términos prácticos, un sistema de aprendizaje de máquina usa datos para ajustar un modelo capaz de realizar una tarea.

Datos + algoritmo + evaluación $\Rightarrow$ modelo

La pregunta principal no es solamente *qué algoritmo usar*, sino:

- ▶ ¿qué queremos predecir o descubrir?,
- ▶ ¿qué datos tenemos disponibles?,
- ▶ ¿cómo sabremos si el modelo funciona bien?,
- ▶ ¿cómo se usará el modelo en un caso real?

Un mapa general del aprendizaje de máquina



En este curso nos enfocaremos principalmente en aprendizaje supervisado y no supervisado.

Aprendizaje supervisado

En aprendizaje supervisado, el conjunto de entrenamiento contiene ejemplos con una respuesta conocida:

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n).$$

- ▶ x_i : características de la observación.
- ▶ y_i : etiqueta o valor que se desea predecir.

Objetivo

Aprender una función aproximada f tal que $f(x)$ produzca buenas predicciones para nuevos datos.

Clasificación

La **clasificación** es un problema supervisado en el que la respuesta pertenece a un conjunto de categorías.

Problema	Clases posibles
Correo electrónico	spam / no spam
Diagnóstico inicial	positivo / negativo
Imagen de una prenda	camiseta / zapato / bolsa / etc.
Texto de opinión	positivo / negativo / neutral

Pregunta clave: ¿a qué clase pertenece esta observación?

Regresión

La **regresión** es un problema supervisado en el que la respuesta es un valor numérico.

Problema	Valor a predecir
Precio de una vivienda	precio en pesos
Demanda de energía	consumo esperado
Ventas de un producto	unidades vendidas
Temperatura futura	grados Celsius

Pregunta clave: ¿qué valor numérico esperamos observar?

Aprendizaje no supervisado

En aprendizaje no supervisado, el conjunto de datos no contiene una respuesta conocida:

$$X_1, X_2, \dots, X_n.$$

El objetivo es descubrir estructura, patrones o representaciones útiles.

- ▶ **Clustering:** agrupar observaciones similares.
- ▶ **Reducción de dimensionalidad:** representar datos complejos en menos variables.
- ▶ **Detección de anomalías:** identificar casos inusuales.

Agrupamiento o clustering

El clustering busca organizar observaciones en grupos sin usar etiquetas previas.

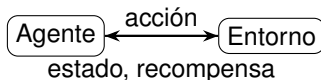
Ejemplos

- ▶ Segmentar clientes por patrones de consumo.
- ▶ Agrupar documentos por temas similares.
- ▶ Identificar tipos de comportamiento en una plataforma digital.
- ▶ Explorar datos antes de construir un modelo supervisado.

Pregunta clave: ¿qué observaciones se parecen entre sí?

Aprendizaje por refuerzo

En aprendizaje por refuerzo, un agente interactúa con un entorno y aprende a tomar decisiones mediante recompensas.



- ▶ Juegos.
- ▶ Robótica.
- ▶ Control automático.
- ▶ Sistemas de recomendación secuenciales.

Nota: lo mencionaremos como contexto, pero no será el foco del curso.

Observaciones, características y etiquetas

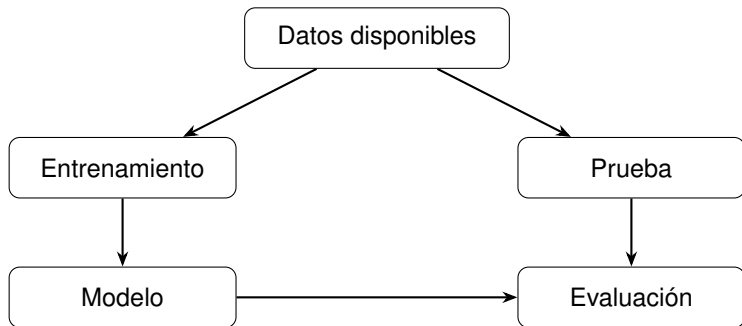
Un conjunto de datos suele organizarse como una tabla.

x_1	x_2	x_3	$\dots$	y
5.1	3.5	1.4	$\dots$	clase A
4.9	3.0	1.4	$\dots$	clase A
6.2	3.4	5.4	$\dots$	clase B

- ▶ Cada fila es una **observación** o instancia.
- ▶ Cada columna x_j es una **característica** o variable explicativa.
- ▶ La columna y es la **etiqueta** o variable objetivo, si existe.

Conjunto de entrenamiento y prueba

Para estimar si un modelo generaliza, separamos los datos en al menos dos partes.



Idea: el modelo se entrena con una parte de los datos y se evalúa con datos que no vio durante el entrenamiento.

Métricas iniciales

La métrica depende del tipo de problema.

Tipo de problema	Métricas comunes
Clasificación	accuracy, precisión, recall, F1-score, matriz de confusión
Regresión	error absoluto medio, error cuadrático medio, R^2
Clustering	inercia, silhouette score, interpretación de grupos

Advertencia

Una métrica alta no siempre implica que el modelo sea útil, justo o adecuado para el contexto real.

Scikit-Learn: una primera idea

Scikit-Learn es una biblioteca de Python para aprendizaje de máquina clásico.

La mayoría de los modelos siguen una interfaz similar:

1. Crear el modelo.
2. Entrenar con `fit()`.
3. Predecir con `predict()`.
4. Evaluar el resultado.

Ventaja

Podemos cambiar de algoritmo manteniendo casi el mismo flujo de trabajo.

Primer ejemplo: cargar datos

Usaremos un conjunto de datos incluido en Scikit-Learn para practicar el flujo básico.

```
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
X = iris.data
y = iris.target

X.head()
```

Discusión:

- ▶ ¿Cuántas observaciones hay?
- ▶ ¿Cuántas características tiene cada observación?
- ▶ ¿Qué representa la variable objetivo?

Iris Dataset

iris setosa



petal sepal

iris versicolor



petal sepal

iris virginica



petal sepal

Source: <https://codesignal.com/learn/courses/intro-to-unsupervised-machine-learning/lessons/exploring-and-visualizing-the-iris-dataset>

Separar entrenamiento y prueba

El primer paso para evaluar un modelo es separar datos de entrenamiento y prueba.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=41
)
```

- ▶ `X_train, y_train`: datos para ajustar el modelo.
- ▶ `X_test, y_test`: datos para evaluar el modelo.
- ▶ `random_state`: permite reproducir el mismo resultado.

Entrenar un primer modelo

Como primer modelo usaremos vecinos más cercanos. La idea es clasificar una observación según las observaciones más parecidas en el conjunto de entrenamiento.

```
from sklearn.neighbors import KNeighborsClassifier  
  
model = KNeighborsClassifier(n_neighbors=3)  
model.fit(X_train, y_train)
```

No memorizar el algoritmo todavía

En esta sesión lo importante es entender el flujo general: datos, entrenamiento, predicción y evaluación.

Predecir y evaluar

Una vez entrenado el modelo, podemos predecir las clases del conjunto de prueba.

```
from sklearn.metrics import accuracy_score, confusion_matrix

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print(accuracy)
print(cm)
```

Preguntas:

- ▶ ¿Qué significa el accuracy obtenido?
- ▶ ¿Qué es y qué información adicional da la matriz de confusión?

Matriz de confusión: Iris dataset

		Clase predicha		
		Setosa	Versicolor	Virginica
Clase real	Setosa	50	0	0
	Versicolor	0	47	3
	Virginica	0	2	48

1. Las filas representan la clase real y las columnas la clase predicha.
2. Los valores en la diagonal son predicciones correctas.

Actividad de laboratorio 1 (es parte de su calificación)

Actividad: modificar el notebook inicial.

1. Cambiar el valor de `n_neighbors`.
2. Probar al menos tres valores diferentes.
3. Registrar el accuracy obtenido en cada caso.
4. Registrar la matriz de confusión obtenida en cada caso.
5. Discutir si el mejor resultado observado necesariamente implica el mejor modelo.

Entrega

Subir a Blackboard el notebook ejecutado (o el código de python) junto con una captura de los resultados principales, y la discusión. **Fecha límite:** Hoy 11:59 pm.

Actividad 2

Actividad: modificar el notebook inicial.

1. Cambiar el valor de `random_state` en `train_test_split(X,y,test_size=0.2,random_state=41)`
2. Probar el valor de 42, y otros dos valores (enteros) diferentes.
3. Registrar el accuracy obtenido en cada caso.
4. Registrar la matriz de confusión obtenida en cada caso.
5. Discutir acerca de los resultados obtenidos.

Entrega

Subir a Blackboard el notebook ejecutado (o el código de python) junto con una captura de los resultados principales y la discusión. **Fecha límite:** Hoy 11:59 pm.

Preguntas guía para el laboratorio

Durante la actividad, responde:

- ▶ ¿Qué columnas contiene el conjunto de datos?
- ▶ ¿Cuál es la variable objetivo?
- ▶ ¿Estamos resolviendo clasificación, regresión o clustering?
- ▶ ¿Qué parte del código entrena el modelo?
- ▶ ¿Qué parte del código evalúa el modelo?
- ▶ ¿Qué limitaciones tiene este primer experimento?

Objetivo: conectar el código con los conceptos vistos al inicio de la sesión.

Cierre de la sesión

Hoy revisamos:

- ▶ aprendizaje supervisado, no supervisado y por refuerzo;
- ▶ clasificación, regresión y agrupamiento;
- ▶ observaciones, características y etiquetas;
- ▶ separación entrenamiento-prueba;
- ▶ primer flujo de trabajo con Scikit-Learn.

Mensaje clave

El tipo de aprendizaje depende de la información disponible y de la pregunta que queremos responder.

Antes de la siguiente sesión

- ▶ **Terminar la práctica 1 y 2 en blackboard.**
- ▶ **Opcional, pero recomendado:** Revisar el capítulo introductorio del libro “*Hands-On Machine Learning with Scikit-Learn and PyTorch*” sobre el panorama del aprendizaje de máquina (Capítulo 1: The Machine Learning Landscape).

Próxima sesión

Flujo de un proyecto de aprendizaje de máquina: definición del problema, preparación de datos y evaluación inicial.

¡Gracias!

¿Preguntas?